Deploy an Azure Resource Manager template in an Automation PowerShell runbook

You can write an Automation PowerShell runbook that deploys an Azure resource by using an Azure Resource Manager template. Templates allow you to use Azure Automation to automate deployment of your Azure resources. You can maintain your Resource Manager templates in a central, secure location, such as Azure Storage.

In this article, we create a PowerShell runbook that uses a Resource Manager template stored in Azure Storage to deploy a new Azure Storage account.

If you don't have an Azure subscription, create a free account before you begin.

Prerequisites

- An Azure Automation account with at least one user-assigned managed identity. For more information, see Using a user-assigned managed identity for an Azure Automation account.
- Az modules: `Az.Accounts`, `Az.ManagedServiceIdentity`, `Az.Resources`, and `Az.Storage`. imported into the Automation account. For more information, see Import Az modules.
- Azure Storage account in which to store the Resource Manager template.
- Azure PowerShell installed on a local machine. See Install the Azure PowerShell Module for information about how to get Azure PowerShell. You'll also need module Az.ManagedServiceIdentity. `Az.ManagedServiceIdentity` is a preview module and not installed as part of the Az module. To install it, run `Install-Module -Name Az.ManagedServiceIdentity`

Assign permissions to managed identities

Assign permissions to the managed identities to do the storage-related tasks in the Runbook.

1. Sign in to Azure interactively using the Connect-AzAccount cmdlet and follow the instructions.

   PowerShell

```powershell
# Sign in to your Azure subscription
$sub = Get-AzSubscription -ErrorAction SilentlyContinue
if(-not($sub))
{
    Connect-AzAccount
}

# If you have multiple subscriptions, set the one to use
# Select-AzSubscription -SubscriptionId <SUBSCRIPTIONID>
```

2. Provide an appropriate value for the variables below and then execute the script.

PowerShell

```powershell
$resourceGroup = "resourceGroup"
$automationAccount = "automationAccount"
$storageAccount = "storageAccount"
$userAssignedManagedIdentity = "userAssignedManagedIdentity"
$storageTemplate = "path\storageTemplate.json"
$runbookScript = "path\runbookScript.ps1"
```

3. Assign the role `reader` to the system-assigned managed identity to execute the cmdlet `Get-AzUserAssignedIdentity`.

PowerShell

```powershell
$SAMI = (Get-AzAutomationAccount -ResourceGroupName $resourceGroup -Name
$automationAccount).Identity.PrincipalId
New-AzRoleAssignment `
    -ObjectId $SAMI `
    -ResourceGroupName $resourceGroup `
    -RoleDefinitionName "Reader"
```

4. Assign the role `Storage Account Contributor` to the user-assigned managed identity for actions against the storage account.

PowerShell

```powershell
$UAMI_ID = (Get-AzUserAssignedIdentity -ResourceGroupName $resourceGroup -Name
$userAssignedManagedIdentity).PrincipalId
New-AzRoleAssignment `
    -ObjectId $UAMI_ID `
    -ResourceGroupName $resourceGroup `
    -RoleDefinitionName "Storage Account Contributor"
```

Create the Resource Manager template

In this example, you use a Resource Manager template that deploys a new Azure Storage account. Create a local file called `storageTemplate.json` and then paste the following code:

JSON

```
{
  "$schema": "https://schema.management.azure.com/schemas/2015-01-
01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "storageAccountType": {
      "type": "string",
      "defaultValue": "Standard_LRS",
      "allowedValues": [
        "Standard_LRS",
        "Standard_GRS",
        "Standard_ZRS",
        "Premium_LRS"
      ],
      "metadata": {
        "description": "Storage Account type"
      }
    },
    "location": {
      "type": "string",
      "defaultValue": "[resourceGroup().location]",
      "metadata": {
        "description": "Location for all resources."
      }
    }
  },
  "variables": {
    "storageAccountName": "[concat(uniquestring(resourceGroup().id), 'standardsa')]"
  },
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "name": "[variables('storageAccountName')]",
      "apiVersion": "2018-02-01",
      "location": "[parameters('location')]",
      "sku": {
          "name": "[parameters('storageAccountType')]"
      },
      "kind": "Storage",
      "properties": {
      }
    }
  ],
  "outputs": {
      "storageAccountName": {
          "type": "string",
          "value": "[variables('storageAccountName')]"
      }
```

```
  }
}
```

Save the Resource Manager template in Azure Files

Use PowerShell to create an Azure file share and upload `storageTemplate.json`. For instructions on how to create a file share and upload a file in the Azure portal, see Get started with Azure Files on Windows.

Run the following commands to create a file share and upload the Resource Manager template to that file share.

PowerShell

```
# Get the access key for your storage account
$key = Get-AzStorageAccountKey -ResourceGroupName $resourceGroup -Name
$storageAccount

# Create an Azure Storage context using the first access key
$context = New-AzStorageContext -StorageAccountName $storageAccount -
StorageAccountKey $key[0].value

# Create a file share named 'resource-templates' in your Azure Storage account
$fileShare = New-AzStorageShare -Name 'resource-templates' -Context $context

# Add the storageTemplate.json file to the new file share
Set-AzStorageFileContent -ShareName $fileShare.Name -Context $context -Source
$storageTemplate
```

Create the PowerShell runbook script

Create a PowerShell script that gets the `storageTemplate.json` file from Azure Storage and deploys the template to create a new Azure Storage account. Create a local file called `runbookScript.ps1` and then paste the following code:

PowerShell

```
param (
    [Parameter(Mandatory=$true)]
    [string]
    $resourceGroup,

    [Parameter(Mandatory=$true)]
    [string]
    $storageAccount,

    [Parameter(Mandatory=$true)]
```

```
    [string]
    $storageAccountKey,

    [Parameter(Mandatory=$true)]
    [string]
    $storageFileName,

    [Parameter(Mandatory=$true)]
    [string]
    $userAssignedManagedIdentity
)

# Ensures you do not inherit an AzContext in your runbook
Disable-AzContextAutosave -Scope Process

# Connect to Azure with user-assigned managed identity
$AzureContext = (Connect-AzAccount -Identity).context
$identity = Get-AzUserAssignedIdentity -ResourceGroupName $resourceGroup `
    -Name $userAssignedManagedIdentity `
    -DefaultProfile $AzureContext
$AzureContext = (Connect-AzAccount -Identity -AccountId $identity.ClientId).context

# set and store context
$AzureContext = Set-AzContext -SubscriptionName $AzureContext.Subscription `
    -DefaultProfile $AzureContext

#Set the parameter values for the Resource Manager template
$Parameters = @{
    "storageAccountType"="Standard_LRS"
    }

# Create a new context
$Context = New-AzStorageContext -StorageAccountName $storageAccount -
StorageAccountKey $storageAccountKey

Get-AzStorageFileContent `
    -ShareName 'resource-templates' `
    -Context $Context `
    -path 'storageTemplate.json' `
    -Destination 'C:\Temp' -Force

$TemplateFile = Join-Path -Path 'C:\Temp' -ChildPath $storageFileName

# Deploy the storage account
New-AzResourceGroupDeployment `
    -ResourceGroupName $resourceGroup `
    -TemplateFile $TemplateFile `
    -TemplateParameterObject $Parameters
```

Import and publish the runbook into your Azure Automation account

Use PowerShell to import the runbook into your Automation account, and then publish the runbook. For information on importing and publishing runbooks in the Azure portal, see Manage runbooks in Azure Automation.

To import `runbookScript.ps1` into your Automation account as a PowerShell runbook, run the following PowerShell commands:

PowerShell

```
$importParams = @{
    Path = $runbookScript
    ResourceGroupName = $resourceGroup
    AutomationAccountName = $automationAccount
    Type = "PowerShell"
}
Import-AzAutomationRunbook @importParams

# Publish the runbook
$publishParams = @{
    ResourceGroupName = $resourceGroup
    AutomationAccountName = $automationAccount
    Name = "runbookScript"
}
Publish-AzAutomationRunbook @publishParams
```

Start the runbook

Now we start the runbook by calling the Start-AzAutomationRunbook cmdlet. For information about how to start a runbook in the Azure portal, see Starting a runbook in Azure Automation.

Run the following commands in the PowerShell console:

PowerShell

```
# Set up the parameters for the runbook
$runbookParams = @{
    resourceGroup = $resourceGroup
    storageAccount = $storageAccount
    storageAccountKey = $key[0].Value # We got this key earlier
    storageFileName = "storageTemplate.json"
    userAssignedManagedIdentity = $userAssignedManagedIdentity
}

# Set up parameters for the Start-AzAutomationRunbook cmdlet
$startParams = @{
    resourceGroup = $resourceGroup
    AutomationAccountName = $automationAccount
    Name = "runbookScript"
    Parameters = $runbookParams
```

```
}

# Start the runbook
$job = Start-AzAutomationRunbook @startParams
```

After the runbook runs, you can check its status by retrieving the property value of the job object `$job.Status`.

The runbook gets the Resource Manager template and uses it to deploy a new Azure Storage account. You can see the new storage account was created by running the following command:

PowerShell

```
Get-AzStorageAccount
```

Next steps

- To learn more about Resource Manager templates, see Azure Resource Manager overview.
- To get started with Azure Storage, see Introduction to Azure Storage.
- To find other useful Azure Automation runbooks, see Use runbooks and modules in Azure Automation.

Last updated on 11/17/2025